

Name	R.K Ayushman Singh
Register No.	261100690014

LAB EXPERIMENT – 2

Importance of Cryptography: HTTP vs HTTPS

1. Aim

To understand why cryptography matters in real network communication by capturing a dummy login attempt over both HTTP and HTTPS using Wireshark, and comparing what an attacker sitting on the same network would actually be able to see in each case.

2. Tools / Software Used

- Kali Linux (attacker machine, running Firefox and Wireshark)
- Docker (used to spin up an Ubuntu container as the target web server, instead of a full VM)
- Ubuntu 24.04 (inside the container)
- Apache2 and PHP (to host the login page)
- OpenSSL (to generate a self-signed certificate for HTTPS)

3. Theory

HTTP sends everything between the browser and the server as plain text. If someone is capturing packets on the same network, the request and response — including form fields like a username and password — can be read directly.

HTTPS is HTTP running on top of TLS (Transport Layer Security). TLS encrypts the data before it leaves the browser, so even if the same packets are captured, all a sniffer sees is unreadable ciphertext. The aim of this lab was to prove this difference practically instead of just reading about it — by setting up one login page and hitting it twice, once over HTTP and once over HTTPS, while Wireshark was running in the background.

4. Lab Setup

Rather than setting up a second virtual machine just to act as a server, I used a Docker container running Ubuntu 24.04 as the target machine. Docker made this a lot faster since the container could be brought up and torn down in seconds, and it still behaves like a real Linux host as far as Apache and networking are concerned.

Two ports were published from the container: **8080** for plain HTTP and **8443** for HTTPS. The container was reachable from Kali at `192.168.56.1`, which is the host-only adapter shared between the Kali VM and the Docker host.

```
docker ps
```

This confirmed the container was up before moving on to installing anything inside it.

5. Setting Up the Login Page

Apache2 and PHP were installed inside the container, and a very simple login page was written that just accepts a username and password over POST and prints a welcome message back. There is nothing complicated in the page on purpose — the point of the lab is the traffic, not the web application.

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $username = $_POST["username"];
    echo "<h2>Login Successful</h2>";
    echo "Welcome, " . htmlspecialchars($username);
}
?>

<!DOCTYPE html>
<html>
<head>
    <title>Login Page</title>
</head>
<body>
    <h1>Cyber Security Lab</h1>
    <h2>Login</h2>
    <form method="POST">
        <label>Username:</label><br>
        <input type="text" name="username" required>
        <br><br>
        <label>Password:</label><br>
        <input type="password" name="password" required>
        <br><br>
        <input type="submit" value="Login">
    </form>
</body>
</html>
```

Fig. 1: The PHP source of the login page used for this lab (Cyber Security Lab login form).

After placing the file in the Apache web root, Apache was started and the page was tested locally with curl before touching it from the browser at all.

```
service apache2 start
curl http://localhost
```

```

</body>
</html>
root@b36699d4c023:/var/www/html# nano index.php
root@b36699d4c023:/var/www/html# service apache2 start
* Starting Apache httpd web server apache2
*
root@b36699d4c023:/var/www/html# curl http://localhost

<!DOCTYPE html>
<html>
<head>
  <title>Login Page</title>
</head>

<body>

```

Fig. 2: Apache started inside the container and the login page confirmed with a local curl request.

Once curl returned the login page HTML, I knew the server side was fine and any issue after this point would be on the network / browser side, not the page itself.



Fig. 3: Port 8080 reachable from Kali — confirms the HTTP port mapping from the container is working before the actual login page was loaded.

6. Capturing the HTTP Login

With the target confirmed reachable, Wireshark was opened on Kali and the interface facing the host-only network (the one carrying traffic to 192.168.56.1) was selected.

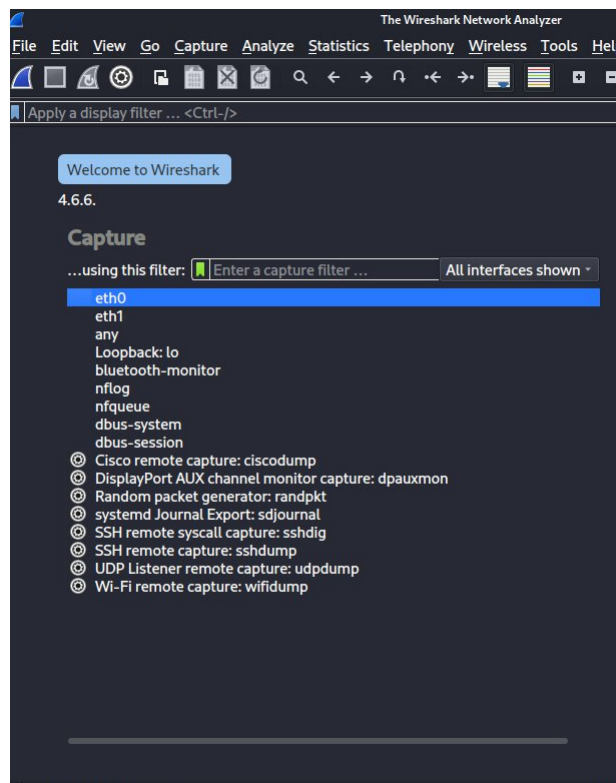


Fig. 4: Wireshark's interface list — the interface used for this capture was selected here before starting.

The capture was started, and the login page was opened at <http://192.168.56.1:8080>. The dummy credentials used were testuser and a made-up password, entered directly into the form.

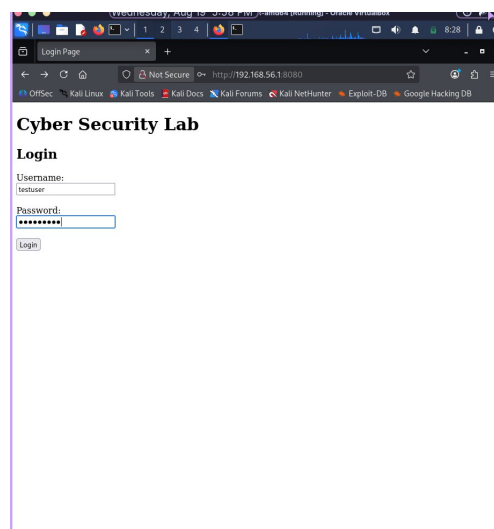


Fig. 5: Login form filled in with the dummy credentials over plain HTTP, before clicking Login.

As soon as the form was submitted, Wireshark showed the GET and POST requests for the page. Filtering on http made it easy to isolate exactly this traffic from everything else on the interface.

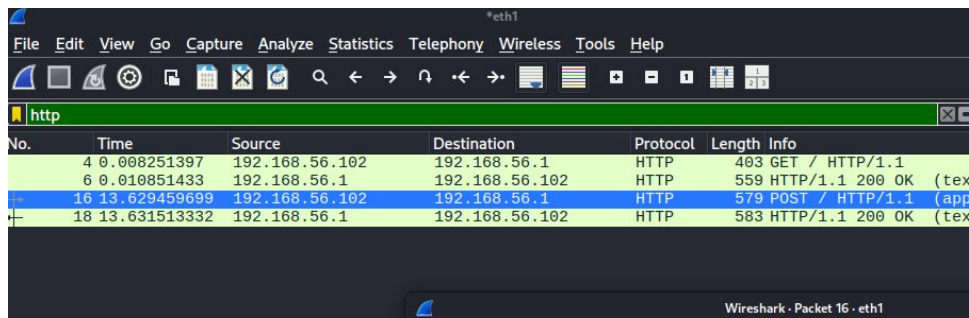


Fig. 6: HTTP filter applied in Wireshark — the POST request for the login is visible in the packet list.

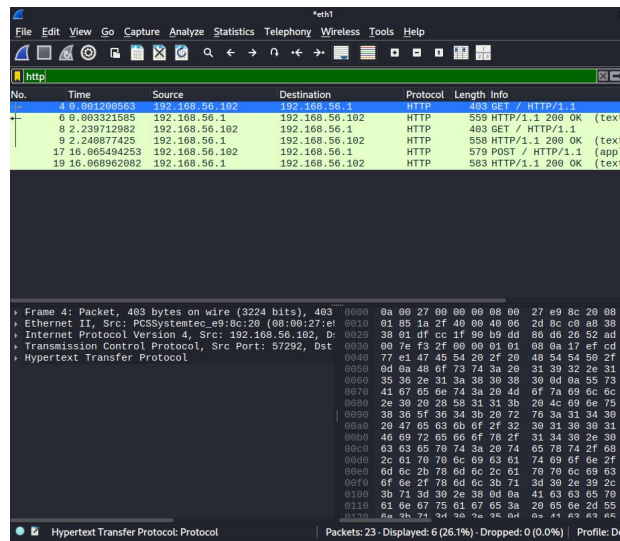


Fig. 7: A closer look at the same HTTP capture, showing the GET/POST/200 OK exchange between Kali and the container.

Opening the POST packet and expanding the HTML Form URL Encoded section showed the username and password exactly as they were typed, sitting right there in the packet — no decoding or special tool needed.

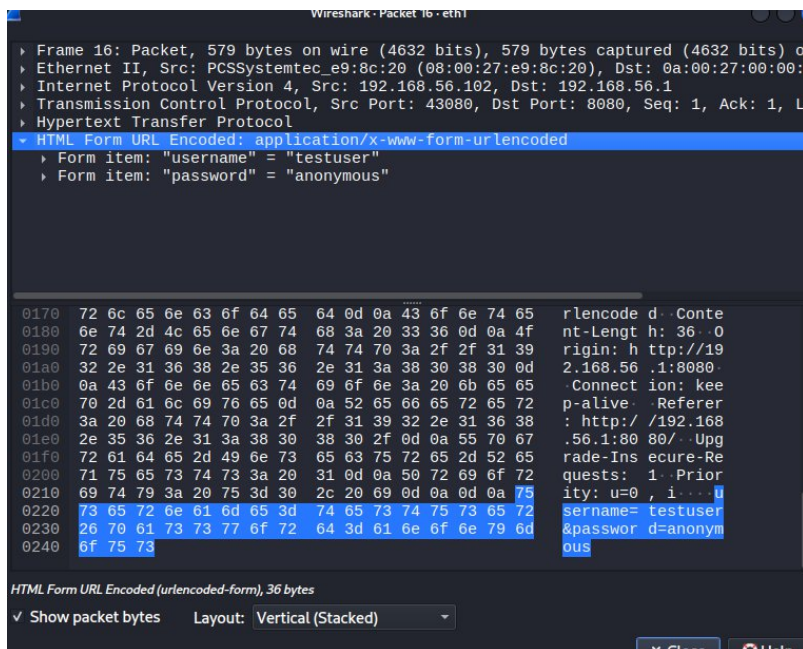
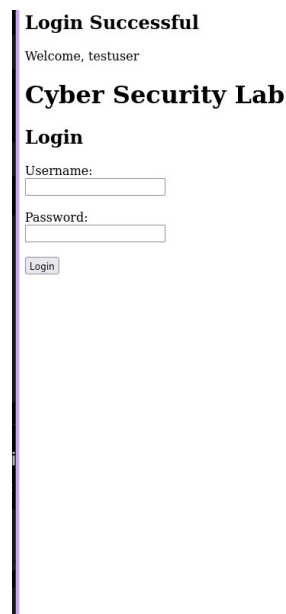


Fig. 8: The POST body decoded by Wireshark. Both the username and password fields are readable in plain text.

This was the main point of the whole exercise — the form data was not obscured in any way. Anyone capturing packets on this network segment could read the login exactly like this.



Login Successful
Welcome, testuser
Cyber Security Lab
Login
Username:
Password:

Fig. 9: The page after a successful HTTP login, confirming the credentials reached the server correctly.

7. Enabling HTTPS on the Same Server

To compare the two protocols fairly, HTTPS was configured on the exact same Apache instance rather than setting up a different server. A self-signed certificate is enough for a lab environment like this, so a proper CA-signed certificate was not required.

```
a2enmod ssl  
a2ensite default-ssl  
apache2ctl configtest
```

apache2ctl configtest returned "Syntax OK", confirming the SSL configuration had no errors before restarting Apache and testing it with curl -k (the -k flag was needed since the certificate is self-signed and not trusted by default).

8. Capturing the HTTPS Login

The same login page was then loaded over HTTPS at <https://192.168.56.1:8443>, and Wireshark was left capturing on the same interface as before.

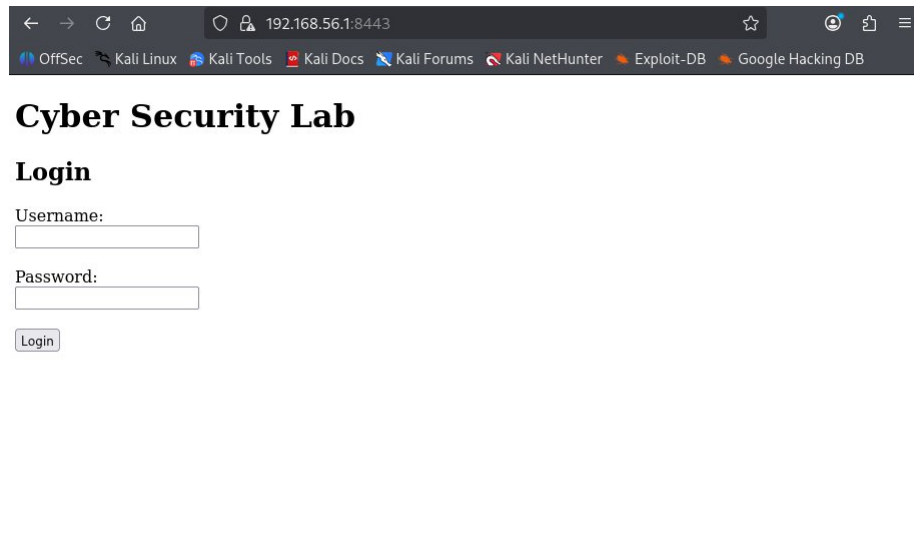


Fig. 10: The login page loaded over HTTPS on port 8443, before entering credentials.

The same dummy username and password were entered again and submitted. This time, filtering the capture on http showed nothing meaningful for the login request — because there is no HTTP request to decode anymore, only encrypted TLS records.

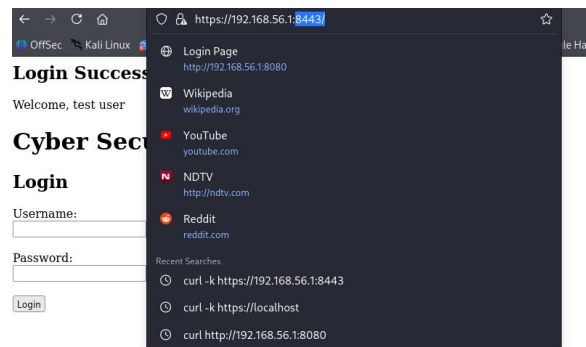


Fig. 11: A successful login over HTTPS at port 8443 — the browser address bar confirms the secure connection.

Switching the Wireshark filter to `tls` showed the full TLS handshake — Client Hello, Server Hello, Change Cipher Spec — followed by a run of Application Data packets, which is where the actual login request is hiding, encrypted.

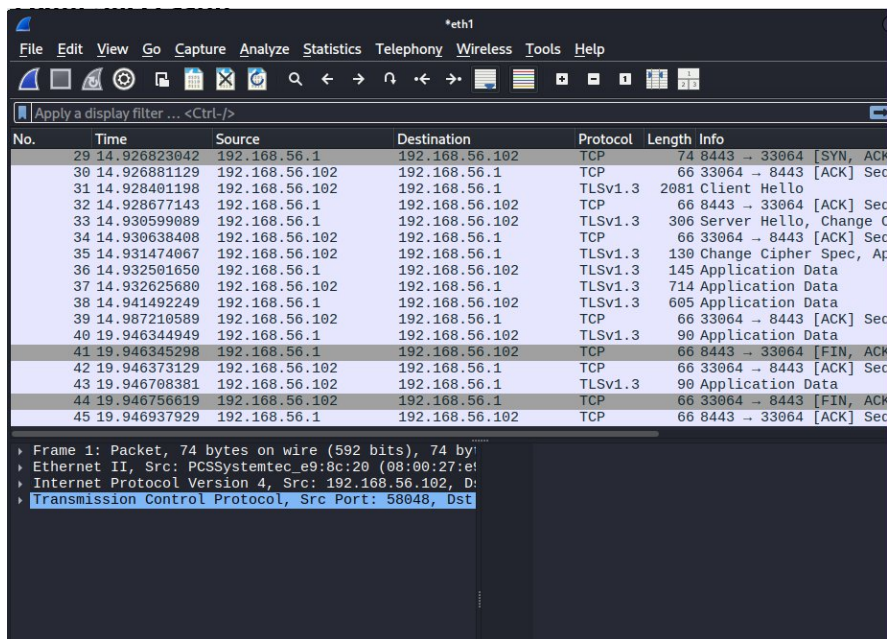


Fig. 12: TLS filter applied in Wireshark — the handshake and the Application Data packets carrying the encrypted login request.

Opening the Client Hello packet confirmed the connection was negotiated using TLS, with none of the form fields visible anywhere in the packet bytes — only handshake parameters and, later, ciphertext.

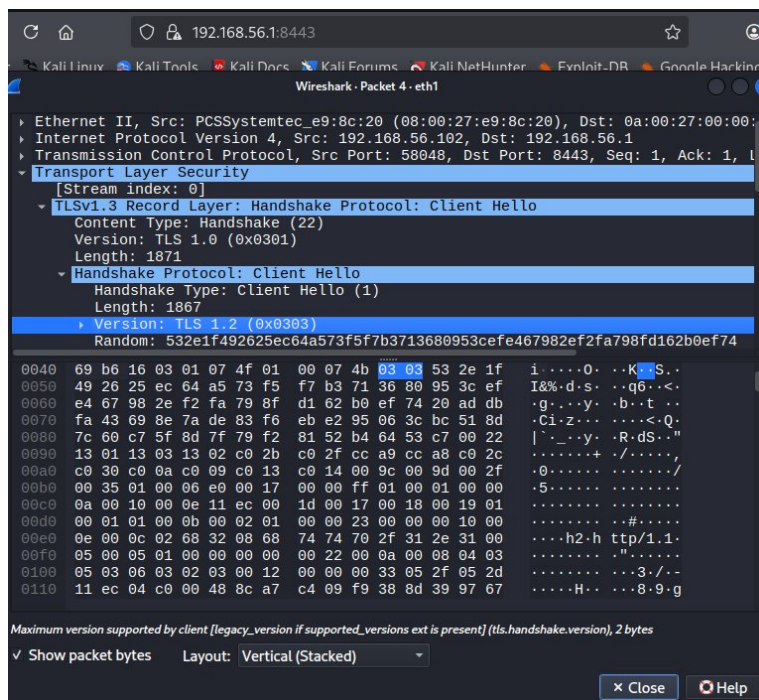


Fig. 13: Client Hello detail from the HTTPS capture — the handshake is visible, but at no point do the actual login credentials appear as plain text.

No matter which packet in this capture was opened, the username and password from the form were never visible. That is the entire value TLS adds — the same credentials, the same network, the same page, but unreadable to anyone sniffing the traffic.

9. Observation

The table below summarises the difference between the two captures taken during this lab.

Parameter	HTTP (port 8080)	HTTPS (port 8443)
Encryption	None	TLS (seen as TLSv1.3)
Protocol seen in Wireshark	HTTP	TLS
Filter used	http	tls
Username / password in capture	Visible as plain text in the form data	Not visible, only Application Data ciphertext
Overall security	Weak, not fit for logins	Strong, safe for logins

The HTTP capture let me read the login straight out of the packet with zero effort. The HTTPS capture, on the same page and the same credentials, gave nothing away — just handshake data and encrypted Application Data records.

10. Result

Login traffic was successfully captured over both HTTP and HTTPS using Wireshark. The HTTP capture exposed the dummy username and password directly in the POST request, while the HTTPS capture protected the same credentials through TLS encryption, with only handshake and ciphertext visible.

11. Conclusion

This lab made the importance of cryptography very concrete for me. Reading about encryption in theory is one thing, but actually watching a password sitting in plain text inside a Wireshark capture — and then watching that same password disappear completely once TLS is switched on — makes the point.

HTTP should never be used to carry anything sensitive, since anyone capturing traffic on the path can read it directly, as shown in this lab. HTTPS, by wrapping the same traffic in TLS, keeps the credentials unreadable even when the exact same packets are captured. This is exactly why login pages, banking sites, and anything handling personal data are expected to run over HTTPS today.

12. Extra practice Screenshots

Altoro Mutual

Sign In | Contact Us | Feedback | Search

ONLINE BANKING LOGIN

PERSONAL

- Deposit Product
- Checking
- Loan Products
- Cards
- Investments & Insurance
- Other Services

SMALL BUSINESS

- Deposit Products
- Lending Services
- Cards
- Insurance
- Retirement
- Other Services

INSIDE ALTORO MUTUAL

- About Us
- Contact Us
- Locations
- Investor Relations
- Press Room
- Careers
- Subscribe

Online Banking Login

Username:

Password:

This connection is not secure. Logins entered here could be compromised.

Learn More

Manage Passwords

Privacy Policy | Security Statement | Server Status Check | REST API | © 2026 Altoro Mutual, Inc.

This web application is open source! Get your copy from GitHub and take advantage of advanced features

The Altoro website is published by HCL Technologies, Ltd. for the sole purpose of demonstrating the effectiveness of HCL products in detecting web application vulnerabilities and website defects. This site is not a real banking site. Similarities, if any, to third party products and/or websites are purely coincidental. This site is provided "as is" without warranty of any kind, either express or implied. HCL does not assume any risk in relation to your use of this website. For more information, please go to <https://www.hcl-software.com/appscan/>.

Copyright © 2008, 2017, IBM Corporation, All rights reserved. Copyright © 2017, 2026, HCL Technologies, Ltd., All rights reserved.

*eth0

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

ip.addr == 65.61.137.117 &&http

No.	Time	Source	Destination	Protocol	Length	Info
440	8.105382873	65.61.137.117	10.0.2.15	HTTP	531	HTTP/1.1 200 OK (text/css)
444	8.286797982	10.0.2.15	65.61.137.117	HTTP	401	GET /style.css HTTP/1.1
448	8.298854224	10.0.2.15	65.61.137.117	HTTP	462	GET /images/home2.jpg HTTP/1.1
582	11.555592727	10.0.2.15	65.61.137.117	HTTP	506	GET /index.jsp?content= HTTP/1.1
645	11.823397902	65.61.137.117	10.0.2.15	HTTP	1332	HTTP/1.1 200 OK (text/html)
663	11.947350191	10.0.2.15	65.61.137.117	HTTP	500	GET /images/p_cards.jpg HTTP/1.1
701	12.212553558	65.61.137.117	10.0.2.15	HTTP	1182	HTTP/1.1 200 OK (JPEG)
757	14.429319754	10.0.2.15	65.61.137.117	HTTP	548	GET /index.jsp?content= HTTP/1.1
792	14.697532811	65.61.137.117	10.0.2.15	HTTP	1390	HTTP/1.1 200 OK (text/html)
795	14.796938524	10.0.2.15	65.61.137.117	HTTP	512	GET /images/p_investments.jpg HTTP/1.1
876	16.031244435	65.61.137.117	10.0.2.15	HTTP	1076	HTTP/1.1 200 OK (JPEG)
1048	84.326007849	10.0.2.15	65.61.137.117	HTTP	521	GET /login.jsp HTTP/1.1
1112	84.620332581	65.61.137.117	10.0.2.15	HTTP	308	HTTP/1.1 200 OK (text/html)
1168	97.538554402	10.0.2.15	65.61.137.117	HTTP	633	POST /doLogin HTTP/1.1
1170	97.800805961	65.61.137.117	10.0.2.15	HTTP	180	HTTP/1.1 302 Found
1172	97.803785449	10.0.2.15	65.61.137.117	HTTP	488	GET /login.jsp HTTP/1.1
1182	98.068981281	65.61.137.117	10.0.2.15	HTTP	412	HTTP/1.1 200 OK (text/html)

Frame 1168: Packet, 633 bytes on wire (5064 bits), captured on interface eth0 at 2023-08-08 10:00:27.538554402

Ethernet II, Src: PCSSystemtec_5a:87:bc (08:00:27:5a:87:bc), Dst: 10.0.2.15

Internet Protocol Version 4, Src: 10.0.2.15, Dst: 65.61.137.117

Transmission Control Protocol, Src Port: 51278, Dst Port: 80, Seq: 302404800, Win: 65535, Len: 633

Hypertext Transfer Protocol

POST /doLogin HTTP/1.1\r\nHost: testfire.net\r\nUser-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:1.9.0.1) Gecko/20100101 Firefox/3.0.1\r\nAccept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8\r\nAccept-Language: en-US,en;q=0.5\r\nAccept-Encoding: gzip, deflate\r\nContent-Type: application/x-www-form-urlencoded\r\nContent-Length: 48\r\nOrigin: http://testfire.net\r\nConnection: keep-alive\r\nReferer: http://testfire.net/login.jsp\r\nCookie: JSESSIONID=77FC51461783DA35811EDCE7DF1391\r\nUpgrade-Insecure-Requests: 1\r\n\r\n

Hypertext Transfer Protocol: Protocol

Packets: 1212 · Displayed: 18 (1.5%) · Dropped: 0 (0.0%) · Profile: Default

```
[Full Request URI: http://testfire.net/doLogin]
File Data: 48 bytes
HTML Form URL Encoded: application/x-www-form-urlencoded
  Form item: "uid" = "sjbsjxbsjbsc"
  Form item: "passw" = "ajxbajxba"
  Form item: "btnSubmit" = "Login"

0000 52 54 00 12 35 00 08 00 27 5a 87 bc 08 00 45 00 RT: 5... 'Z...E
0010 02 6b 3a 24 40 00 40 06 27 a8 0a 00 02 0f 41 3d .k:$@_@ '.....A
0020 89 75 c8 4e 00 50 42 ff 5d 94 2c d0 c9 58 50 18 .U N PB ],...XP
0030 ff ff d9 1e 00 00 50 4f 53 54 20 2f 64 6f 4c 6f .....PO ST /doLo
0040 67 69 6e 20 48 54 54 50 2f 31 2e 31 0d 0a 48 6f gin HTTP /1.1..Ho
0050 73 74 3a 20 74 65 73 74 66 69 72 65 2e 6e 65 74 st: test fire.net
0060 0d 0a 55 73 65 72 2d 41 67 65 6e 74 3a 20 4d 6f .User-Agent: Mo
0070 7a 69 6c 6c 61 2f 35 2e 30 20 28 58 31 31 3b 20 zilla/5.0 (X11;
0080 4c 69 6e 75 78 20 78 38 36 5f 36 34 3b 20 72 76 Linux x86_64; rv
0090 3a 31 34 30 2e 30 29 20 47 65 63 6b 6f 2f 32 30 :140.0) Gecko/20
00a0 31 30 30 31 30 31 20 46 69 72 65 66 6f 78 2f 31 100101 Firefox/1
00b0 34 30 2e 30 0d 0a 41 63 63 65 70 74 3a 20 74 65 40.0. Ac cept: te
00c0 78 74 2f 68 74 6d 6c 2c 61 70 70 6c 69 63 61 74 xt/html, applicat
00d0 69 6f 6e 2f 78 68 74 6d 6c 2b 78 6d 6c 2c 61 70 ion/xhtm l+xml,ap

p: 1168 - Time: 97.538554402 - Source: 10.0.2.15 - Destination: ... Info: POST /doLogin HTTP/1.1 (application/x-www-form-urlencoded)
Show packet bytes Layout: Vertical (Stacked)
[Close] [Help]
```

12. Precautions

- Only dummy credentials were used at every stage; no personal or real account details were entered.
- The entire experiment was carried out inside an isolated lab network (Kali VM and a local Docker container), with nothing exposed to the internet.
- Captured packets were used strictly for this lab exercise and were not saved or shared beyond this report.
- The self-signed certificate used for HTTPS was only for demonstrating encryption and is not meant for production use.